Motion Detection Using Image Filtering

Caleb Bloch
EECE 5639 Computer Vision (Spring 2026)
Northeastern University
Boston, MA, USA

***Abstract*—This project detects motion in an image sequence captured by a camera by measuring temporal greyscale value changes at each pixel. For each experiment setting, I compute a temporal derivative across time, take the magnitude, and threshold it to produce a binary motion mask. I then overlay the mask on the original frame to visualize detections. Then, I compare a simple central-difference temporal derivative against a temporal derivative-of-Gaussian filter across multiple temporal scales, and I evaluate 2D spatial smoothing applied before temporal differencing using box and Gaussian filters. Finally, I use an adaptive per-frame threshold by estimating noise from the temporal-gradient magnitudes.**

I. Introduction

Motion detection in a fixed camera video can be approached by looking for pixels whose greyscale values change over time. Background pixels should remain close to constant (ideally), while pixels that belong to a moving foreground object should change more strongly as the object passes through the scene. An approach is to compute a temporal derivative at each pixel and threshold the magnitude to create a motion mask. In real sequences, noise and other factors can cause false positives, so filtering and thresholding choices can affect mask quality. In this project, I implement filtering based motion detection and evaluate how temporal scale and spatial smoothing can affect the results.

II. Method

A. Data and Preprocessing

The input is a sequence of JPG frames from the "RedChair" dataset. Each frame is read in grayscale and stored as `float32` to make filtering operations easier and to avoid issues when subtracting frames. In the example figures shown in this report (frame 63), the dominant moving foreground object is a person walking through the scene.

B. Spatial Smoothing

In Part 2 of the project, I apply a 2D spatial smoothing filter to each frame before computing the temporal derivative. For Part 1, spatial smoothing was disabled to isolate the temporal derivative. I tested 3×3 and 5×5 box filters and 2D Gaussian smoothing for multiple s_σ values. For Gaussian smoothing, the kernel size is chosen as $6s_\sigma + 1$ (rounded to an odd integer), which covers a $3s_\sigma$ radius.

C. Temporal Derivative Filters

The temporal derivative is computed per pixel after spatial smoothing.

- **Simple central difference:** For frame index t, the temporal derivative is approximated using the central difference filter $0.5[-1, 0, 1]$. This uses the previous and next frames.
- **Temporal derivative-of-Gaussian:** I used a 1D derivative-of-Gaussian kernel in time with radius $r = \lceil 3t_\sigma \rceil$ and applied it along the time axis at each pixel. This requires r frames on both sides of t.

D. Magnitude and Thresholding

After computing the temporal derivative image D_t, I take its magnitude: $M_t(x, y) = |D_t(x, y)|$. To convert M_t into a binary motion mask, I use an adaptive threshold based on a per-frame estimate of background noise. Assuming most pixels are background, I estimate the noise scale using the standard deviation:

$$\sigma_t = \text{std}(M_t) \tag{1}$$

$$T_t = K_{MULT} \times \sigma_t \tag{2}$$

I used $K_{MULT} = 3.0$. The binary mask is defined by $B_t(x, y) = 1$ if $M_t(x, y) > T_t$, else 0.

E. Output / Visualization

I set the script to save every processed frame (`SAVE_EVERY = 1`) to view directly subsequent frames. This allows for a better grasp of mask evolution compared to a larger skip interval.

III. Results and Observations

A. Temporal Filtering Results (Part 1)

Applying the temporal derivative directly without spatial smoothing resulted in extremely noisy motion masks. Both methods struggled with high-frequency frame-to-frame pixel variation. As seen in Fig. 1, the simple difference produced heavy speckle noise. Because I am thresholding temporal change, the mask is usually strongest on moving edges (leading/trailing boundaries), so it does not always fill the entire person.

B. Spatial Smoothing Results (Part 2)

Spatial smoothing had a large positive impact.

- **Box Filters:** Reduced noise significantly; the 5×5 performed slightly better than the 3×3.
- **Gaussian Smoothing:** Provided the best results. $s_\sigma = 2.0$ (Fig. 2) was the cleanest result, with masks that looked natural and rounded.

C. Combination Results

I tested $t_\sigma = 1.0$ and $s_\sigma = 1.0$. This combo did not clearly outperform simple temporal differencing with stronger spatial smoothing. The setting still produced jittery masks around the legs, which are closer in color to their background. This suggests that spatial noise/texture is the primary limit for this dataset.

IV. Discussion

Spatial smoothing was the critical factor for reducing false positives. Thresholding using $T_t = 3 \times \text{std}(M_t)$ adapts automatically across settings. In frames with little motion, $\text{std}(M_t)$ can be small, so mild flicker can still create background speckle; increasing K_{MULT} would reduce false positives but could miss weaker motion.

V. Conclusion

Spatial smoothing is a critical preprocessing step for this dataset. While temporal filtering helped slightly, a strong spatial Gaussian blur ($s_\sigma = 2.0$) produced the most natural masks. The adaptive thresholding mechanism successfully automated the detection sensitivity for frames with objects in motion.

References

[1] Project 1 Handout: Motion Detection Using Image Filtering, EECE 5639: Computer Vision, Northeastern University, Spring 2026.

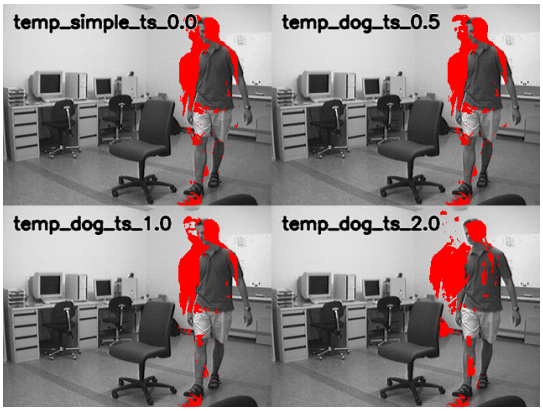


Fig. 1. Temporal comparison (Part 1) at frame 63 showing noise without spatial smoothing.



Fig. 2. Spatial comparison (Part 2) at frame 63 showing clean detection with $s_\sigma = 2.0$.